PaperToSkill: Turning Research Papers into Portable Agent Skills

Anonymous submission

Abstract

Many LLM and agent papers introduce workflows that could help ordinary users and agent builders, but those workflows are usually described for human reading rather than direct reuse. Summaries compress the idea, but often discard the procedural details, validation checks, assumptions, and failure branches needed to apply the method. We introduce PaperToSkill, a paper-to-skill conversion workflow that turns source-anchored paper notes into compact, human-editable agent skills. Each skill records the paper's contribution, use conditions, workflow, validation checks, failure cases, transfer notes, and source anchors. In a four-paper benchmark over AI Scientist-v2, Reflexion, AIDE, and Toolformer, PaperToSkill generates skills that score 20/20 on deterministic structural rubrics, preserve more deterministic operational coverage than generic-summary and abstract-only baselines, remain under a 1200-word compactness budget, and substantially reduce deterministic input-token proxy size relative to full extracted papers. These results support PaperToSkill as a reproducible conversion layer from curated paper notes to portable skills. All four live-transfer saved-response sets are collected and scored under a deterministic output-contract evaluator; these saved response files are not human semantic fidelity or downstream execution-outcome evidence. A first single-run real-reuse stress test over eight AIDE, SWE-agent, Reflexion, and SnapATAC2 paper-task rows is complete, but the outcome is mixed rather than confirmatory: one SWE task favors PaperToSkill, Reflexion ties Summary, and AIDE, SWE-T1, and SnapATAC2 expose timeout, patch-application, and artifact completion failures. We therefore treat these rows as downstream boundary evidence, not aggregate effectiveness.

Introduction

Using LLMs is becoming a practical skill for researchers, builders, and non-expert users. At the same time, many of the best ways to use LLM agents are introduced in papers whose methods are difficult to operationalize. A paper may describe a search policy, memory loop, validation stage, or interface contract, but a user who wants to reuse the contribution must translate prose into an agent workflow by hand.

PaperToSkill studies a simple hypothesis: papers can be converted into compact, editable skills that preserve enough procedural knowledge for agents and users to reuse the main contribution. We use "skill" to mean a natural-language operational artifact with an entry-point `SKILL.md`, similar to formats used by agent harnesses that load task-specific instructions. Unlike a summary, a skill is not only explanatory. It should specify when to use the method, which inputs are required, which steps to execute, how to validate progress, which failures to watch for, and how to adapt the instructions across harnesses.

This paper takes an artifact-first view. PaperToSkill is not presented as a fully automatic PDF understanding system. The current implementation converts curated, source-anchored paper notes into skills and source maps. This narrow scope is intentional: it lets us evaluate whether the conversion layer preserves operational knowledge before claiming end-to-end PDF automation.

Our current contributions are: (1) a paper-to-skill schema for operationalizing research contributions; (2) a deterministic extraction scaffold that emits `SKILL.md` plus source maps; (3) deterministic extracted-text-to-note scaffolds evaluated on Toolformer and AIDE; (4) a four-paper benchmark using AI Scientist-v2, Reflexion, AIDE, and Toolformer; (5) deterministic/offline evaluations for structure, context coverage, compactness, source grounding, and transfer readiness; and (6) a first single-run real-reuse stress test that records both positive and failed Summary-vs-PaperToSkill outcomes without promoting them into an aggregate downstream-success claim.

Related Work

PaperToSkill is motivated by automated research and agent workflow systems. AI Scientist-v2 uses agentic tree search and staged experiment management for automated scientific discovery (Lu et al. 2025). AIDE frames ML engineering as search over code solutions (AIDE). Reflexion stores verbal feedback in episodic memory to improve later attempts without weight updates (Shinn et al. 2023). Toolformer shows how language models can generate and filter tool-use examples for API calls (Schick et al. 2023). Skill-library and interface work such as Voyager and SWE-agent also demonstrate that reusable skills and agent-facing interfaces can determine whether a method transfers beyond its original implementation (Wang et al. 2023; Yang et al. 2024). Paper2Agent is the closest concurrent system: it converts papers and associated

codebases into MCP servers with tools, resources, prompts, tests, and an interactive paper-agent layer (Miao et al. 2025). AgenticSciML shows a related use of specialized agents, structured debate, method memory, and evolutionary search for scientific machine-learning discovery (Jiang and Karniadakis 2026).

These systems show that papers can become active computational artifacts. PaperToSkill targets a different conversion layer: compact, human-editable skills that preserve operational knowledge, validation checks, failure branches, transfer notes, and source boundaries without requiring an MCP server or a runnable paper codebase.

Method

PaperToSkill represents each converted paper as a skill package. The main file is `SKILL.md`; optional references include a source map that links generated skill bullets to source note sections and line spans. The schema contains identity, central contribution, use conditions, required artifacts, workflow steps, validation checks, failure cases, transfer notes, and source notes that distinguish source-backed material from inferred guidance.

The deterministic extractor reads a source-anchored paper note, normalizes Markdown sections and list items, collects candidates from abstract, methods, experiments, limitations, and transfer sections, then emits a compact skill. Candidate limits keep the artifact within a practical context budget while preserving method, validation, and failure coverage. The extractor also writes a `references/source_map.json` file so later audits can inspect which source section supports each generated instruction.

The text-to-note scaffold is an earlier preprocessing step. It operates on newline-numbered extracted text, detects broad paper regions, selects line windows for methods, experiments, and limitations, and preserves line anchors. For two-column extracted text, it keeps raw line spacing and selects the keyword-bearing column while retaining the original source line range. The output is explicitly marked as an audit scaffold.

Experimental Setup

We evaluate four real agent-method papers: AI Scientist-v2, Reflexion, AIDE, and Toolformer. For each paper, we create a curated source-anchored note from the extracted paper text, generate a PaperToSkill skill, and compare it against two context baselines: a generic summary and an abstract-only context. For transfer ablation, we compare the full skill against the same skill with `Transfer Notes` removed and against the generic summary.

The metrics are deterministic: skill rubric score, context coverage, source-span validation, offline transfer-readiness, word count under a 1200-word compactness budget, character-based token/cost proxy, and a local tokenizer-aware input proxy using `o200k_base`. For saved model-ablation responses, we additionally compute a local output-token proxy. These metrics are reproducible gates. They do not replace live agent execution, real billing records, invoice evidence, success-per-dollar accounting, or completed human fidelity annotation. The current package uses local token accounting over input-token and saved-response output-token proxies. We also prepare model-ablation prompt packets for Claude Opus 4.8, a GPT-family slot requested as GPT 5.5, and a DeepSeek slot, and include a protocol-aware runner and response evaluator. The current two-case protocol has 6/6 saved and scored rows across Claude, GPT-family, and DeepSeek slots. This is saved-response output-contract evidence only; it is not live downstream task success, provider billing, or a broad model-quality result. Provider/model availability failures are logged separately from model-quality results.

The primary downstream experiment is the real-reuse protocol in Table 1. It contains eight original-style *paper-task* rows across AIDE, SWE-agent, Reflexion, and SnapATAC2. Each row fixes the source paper, task input/output shape, metric family, reported reference boundary, and the two main conditions: Summary and PaperToSkill. At this revision, all eight rows have one GPT-family run with scored Summary and PaperToSkill outputs. The AIDE rows use an official Kaggle Spaceship Titanic training file with a locked local validation split; the SnapATAC2 rows use prepared official miniature fixtures and are scored local dry-run tasks rather than full paper reproductions. These rows complete the first single-run pass over the real-reuse table, but they do not by themselves establish aggregate advantage over Summary.

Results

The completed results below are deterministic/offline quality and grounding evidence for the current PaperToSkill artifacts, plus first-pass real-reuse execution evidence. AIDE-T1 and AIDE-T2 are scored for one GPT-family run on a locked Kaggle Spaceship Titanic validation split; both Summary and PaperToSkill produce scripts that exceed the 60-second scoring budget and score 0.000. SWE-T1 is scored for one GPT-family run on a locked SWE-Bench Lite SQLFluff instance: both Summary and PaperToSkill produce patches that fail to apply and score 0.000. SWE-T2 is scored for one GPT-family run on a locked SWE-Bench Verified-style Astropy instance: the Summary patch fails to apply and scores 0.000, while the PaperToSkill patch applies, passes the hidden target tests, and scores 1.000. In the real-reuse table, REF-T1 and REF-T2 are also scored for one GPT-family run: both Summary and PaperToSkill reach 1.000 on the locked Reflexion QA and retry fixtures. SNAP-T1 and SNAP-T2 are scored for one GPT-family run over prepared official miniature fixtures. PaperToSkill scores above Summary on these two SnapATAC2 dry-run tasks (0.500 vs. 0.000 and 0.400 vs. 0.200), but all SNAP scores remain below the success threshold, mainly because the responses did not produce complete runtime, memory, and quality artifacts. Across the eight filled tasks, the runner, scorer, and table-update paths are live. The outcome is mixed: SWE-T2 is a positive PaperToSkill row, REF shows no advantage because both conditions solve the fixtures, and AIDE/SWE-T1/SNAP primarily expose budget, patch-application, and artifact-completion failure boundaries. These results do not establish aggregate advantage over Summary. All four generated skills score 20/20 on deterministic skill rubrics. On

Table 1: Main real-reuse experiment table. Filled scores come from local raw rows; future reruns or additional rows may be added. Original-paper scores are reported references unless the same data, input/output, metric, budget, and runtime setting are reproduced locally.

Task	Paper	Domain	Input	Output	Metric	Reference	Summary	PaperToSkill	Status
AIDE-T1	AIDE	ML engineering	Kaggle-style dataset + metric	Runnable solution/submission	validation_score	Reported AIDE ref.	0.000	0.000	Scored (GPT-family)
AIDE-T2	AIDE	ML engineering	Weak ML script + feedback	Improved script + trajectory	best_node_score	Reported AIDE ref.	0.000	0.000	Scored (GPT-family)
SWE-T1	SWE-agent	Software engineering	Repo issue + tests	Patch + test log	resolved	Reported SWE-agent ref.	0.000	0.000	Scored (GPT-family)
SWE-T2	SWE-agent	Software engineering	Failing test + repo	Focused patch + verification	tests_passed	Reported SWE-agent ref.	0.000	1.000	Scored (GPT-family)
REF-T1	Reflexion	Reasoning / QA	Multi-hop QA + feedback	Final answer + reflection trace	exact_match_or_f1	Reported Reflexion ref.	1.000	1.000	Scored (GPT-family)
REF-T2	Reflexion	Decision / programming	Failed attempt + checker feedback	Corrected second attempt	second_attempt_success	Reported Reflexion ref.	1.000	1.000	Scored (GPT-family)
SNAP-T1	SnapATAC2	Single-cell omics	Small single-cell dataset	Pipeline + embedding artifacts	runtime_memory_quality	Reported SnapATAC2 ref.	0.000	0.500	Scored (GPT-family)
SNAP-T2	SnapATAC2	Single-cell omics	Single-cell labels/proxy task	Clustering/marker artifacts	ari_nmi_runtime_memory	Reported SnapATAC2 ref.	0.200	0.400	Scored (GPT-family)

Table 2: Main deterministic/offline PaperToSkill results. Coverage scores are task-rubric scores; transfer readiness is an offline artifact-readiness metric.

Paper	Rubric	Skill	Summary	Abstract	ΔSum.	Transfer	Support	Words
AI Scientist-v2	20/20	7.867/9	1.733/9	1.2/9	6.134	10/10	0.938	782
Reflexion	20/20	8.267/9	3.483/9	2.533/9	4.784	10/10	1.000	479
AIDE	20/20	9.1/10	1.916/10	1.333/10	7.184	10/10	1.000	927
Toolformer	20/20	8.9/10	2.5/10	1.534/10	6.400	10/10	1.000	943

Table 3: Transfer-note ablation. Removing only `Transfer Notes` lowers offline readiness across all curated skills.

Paper	Full	No Transfer
AI Scientist-v2	10/10	7.6/10
Reflexion	10/10	7.6/10
AIDE	10/10	7.6/10
Toolformer	10/10	7.6/10

context coverage, the generated skills score 7.867/9 for AI Scientist-v2, 8.267/9 for Reflexion, 9.1/10 for AIDE, and 8.9/10 for Toolformer. The generic-summary and abstract-only baselines score substantially lower across all four cases (Table 2).

The generated skills remain compact: 782 words for AI Scientist-v2, 479 words for Reflexion, 927 words for AIDE, and 943 words for Toolformer, all under the 1200-word budget. Source validation finds support rates of 0.938, 1.0, 1.0, and 1.0 with zero invalid line ranges. The one weak AI Scientist-v2 span is recorded as a boundary case rather than treated as fully verified.

The tokenizer-aware context proxy shows the same compactness pattern relative to full paper contexts. Under `o200k_base`, generated skills use between 2.39% and 9.65% of the input tokens required by full extracted paper text (Table 4). A separate character-proxy report preserves the earlier $\lceil \text{characters}/4 \rceil$ estimate for reproducibility. We interpret both as coverage-preserving compression relative to full-paper context, not as provider billing evidence or a claim that skills are the shortest possible context.

Transfer-note ablations show a consistent offline readiness pattern. Full skills score 10/10 on the readiness metric across all four curated papers. Removing `Transfer Notes` lowers readiness to 7.6/10 in all four cases, while generic summaries score between 1.2/10 and 2.25/10. This supports the narrower claim that transfer notes encode artifact-readiness signals. It does not yet prove improved live cross-harness success.

The live-transfer saved-response evaluation covers all four paper packets. AI Scientist-v2, Reflexion, AIDE, and Toolformer each have six saved responses across Codex-style and Claude-style harness prompts and three context variants. The aggregate evaluator reports 24 total rows, 24 scored rows, 0 pending rows, and average normalized score 1.0. AI Scientist-v2, Reflexion, and AIDE rows score 11/11 each; Toolformer rows score 9/9 each. The AIDE run includes one provider fallback row where `claude-opus-4-8` closed the connection and `claude-opus-4-7` succeeded. These are deterministic output-contract scores over saved response files, not human-rated semantic fidelity or downstream execution-outcome rates.

The automatic note scaffold is evaluated separately on Toolformer and AIDE. The Toolformer auto-note-derived skill scores 20/20 on the deterministic rubric, 9.3/10 on context coverage, 10/10 on offline transfer readiness, and 1.0 source-span support with zero invalid ranges. The AIDE auto-note-derived skill scores 20/20, 8.467/10 on context coverage, 9.5/10 on offline transfer readiness, and 1.0 source-span support with zero invalid ranges (Table 5). These results support the narrower claim that extracted text can seed auditable note scaffolds. The local pipeline also includes a direct-PDF smoke path through `pdftotext -layout` that records the generated text source in the manifest, but this does not establish robust arbitrary-PDF automation.

We additionally compare PaperToSkill against Paper2Agent at the artifact and workflow level using the Paper2Agent paper text and current PaperToSkill artifacts. The bounded comparison covers required inputs, generated artifact type, setup burden, validation checks, failure handling, source traceability, and runtime dependency. All seven criteria are source-backed and ready in the local comparison table. The result clarifies positioning: Paper2Agent produces MCP servers for papers with associated codebases, while PaperToSkill produces portable natural-language skills with

Table 4: Local tokenizer-aware context-size proxy using the `o200k_base` tokenizer. Counts are not provider bills, output-token costs, or success-per-dollar measurements.

Paper	Full Paper Tokens	Skill Tokens	Skill/Full	Reduction
AI Scientist-v2	45212	1079	0.0239	97.61%
Reflexion	16414	703	0.0428	95.72%
AIDE	13312	1285	0.0965	90.35%
Toolformer	20365	1255	0.0616	93.84%

Table 5: Automatic extracted-text note scaffolds are evaluated separately from curated-note cases.

Paper	Input	Rubric	Coverage	Transfer	Support	Words
Toolformer	Curated note	20/20	8.9/10	10/10	1.000	943
Toolformer	Auto note scaffold	20/20	9.3/10	10/10	1.000	1179
AIDE	Curated note	20/20	9.1/10	10/10	1.000	927
AIDE	Auto note scaffold	20/20	8.467/10	9.5/10	1.000	998

explicit source and failure boundaries. This comparison does not run Paper2Agent, deploy an MCP server, or establish an end-to-end baseline performance result.

Usage Examples and Model Ablations

The repository includes usage examples for three practical workflows: loading a generated skill in a Codex-style harness, generating an automatic note scaffold from extracted text or a smoke-tested PDF source, and building model-ablation prompts. The model-ablation protocol produces a prompt grid for Claude Opus 4.8, a GPT-family slot requested as GPT 5.5, and a DeepSeek slot. In the current protocol, the saved-response evaluator reports 6 total rows, 6 scored rows, 0 pending rows, and average normalized score 1.0. The GPT-family protocol refresh completed both rows with `gpt-5.5`. The DeepSeek run completed both rows with `deepseek-v4-flash`. The latest Claude protocol refresh used the correct Anthropic Messages path but was blocked by provider HTTP 502; the scored Claude rows come from previously saved response files. A local output-token proxy over all six saved model-ablation responses counts 9,594 `o200k_base` output tokens. This 6/6 saved-response score is not live downstream task success, provider billing, live-invoice, success-per-dollar evidence, or a broad model-quality result.

The bounded AI-Scientist-v2 integration path is complete for the local marker smoke and one full live run. The smoke report records a complete marker-contract response, and the full-run handoff records one completion directory. The AI-Scientist-v2-generated synthetic benchmark ties skill and full-excerpt task-success rates at 0.80 while the skill uses fewer tokens (86.2 versus 113.2); abstract, generic-summary, and no-context baselines score 0.20, 0.00, and 0.00. A retrieval-depth sensitivity branch reaches 1.00 only when K=all. This is bounded integration and synthetic sensitivity evidence, not human fidelity, real-data validation, or broad live research-task success.

Discussion

The main result is not that PaperToSkill "understands" papers in the broadest sense. The result is that a structured paper-note-to-skill pipeline can preserve operational knowledge that summaries discard. The skills include workflow steps, validation checks, failure cases, and source anchors, which are exactly the parts needed when a user wants an agent to apply a paper rather than merely explain it.

The benchmark also reveals useful failure modes. During the AIDE case, the initial extractor capped workflow, validation, and failure candidates too aggressively, which dropped data-preview and LLM-cost content. Earlier phases also exposed title parsing, source-audit mapping, and source-span line-counting bugs. These records are the type of failure branches PaperToSkill should preserve: not just final successes, but the ways extraction, evaluation, or external dependencies can lose important operational content.

The real-reuse rows add a harsher version of the same lesson. The AIDE rows show that a method skill can still fail when the generated script exceeds a locked scoring budget. SWE-T1 shows that software-engineering reuse can fail before tests run if a patch cannot be applied cleanly. The SnapATAC2 rows show that partial pipelines are not enough when the scorer requires complete runtime, memory, and quality artifacts. These failures point to the next method pressure points: budget contracts, patch/application constraints, required artifact manifests, and recovery instructions should be explicit when a source paper's method is converted into a reusable skill.

Limitations

The current work has several important limits. First, the main benchmark converts curated notes rather than arbitrary PDFs; automatic note scaffolds have only been retained on Toolformer and AIDE extracted text. Second, the metrics are deterministic and partly lexical, so they can over-credit exact matches or under-credit valid paraphrases. Third, live-transfer saved-response coverage is complete for the current prompt-packet protocol, but the scorer is determinis-

tic and contract-based rather than a human semantic-fidelity or live execution-outcome evaluation. Fourth, the bounded AI-Scientist-v2 smoke and full live run is complete, but its positive result is synthetic and should not be read as broad live task success. Fifth, human-fidelity packets, a reviewer handoff guide, a stricter blank annotation template, and a summary script are prepared, but no independent annotations have been completed. Sixth, compactness is measured by word count, deterministic character proxy, local tokenizer-aware input-token proxy, and local output-token proxy for saved model responses, not by provider-specific prices, live invoices, or live success per dollar. Seventh, the real-reuse rows are single-run GPT-family measurements, and original-paper reference scores are reported references unless the same data, input/output contract, metric, budget, and runtime setting are reproduced locally. Several rows fail because generated scripts or patches do not finish within the locked budget or do not produce complete scorer artifacts, so they should be read as boundary evidence rather than a broad downstream effectiveness claim.

Conclusion

PaperToSkill turns paper-derived procedural knowledge into portable, human-editable skills. In a four-paper benchmark, generated skills are compact, source-grounded, structurally valid, and more operationally complete than short summary baselines under deterministic evaluation. A first eight-row real-reuse stress test is now complete, but it is mixed and failure-heavy rather than a proof of aggregate downstream effectiveness. The next stage is to add human fidelity review, repeat and expand real-reuse runs, extend the bounded Paper2Agent artifact comparison into a real executable MCP baseline if resources permit, and use the real-reuse failure modes to refine budget, artifact, and failure-recovery contracts.

References

AIDE. 2025. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138.

Jiang, Q.; and Karniadakis, G. 2026. AgenticSciML: Collaborative Multi-Agent Systems for Emergent Discovery in Scientific Machine Learning. *npj Artificial Intelligence*. Article in press.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2025. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. arXiv:2504.08066.

Miao, J.; Davis, J. R.; Zhang, Y.; Pritchard, J. K.; and Zou, J. 2025. Paper2Agent: Reimagining Research Papers As Interactive and Reliable AI Agents. arXiv:2509.06917.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.